

Table of Contents

Project Overview:	3
Implementation Steps:	3
1. Data Collection	3
2. Data Cleanup and Preprocessing	4
3. Model Training	4
4. Model Evaluation	6
5. User Interface Development	8
Final Outcomes & Learnings	10
1. Key Achievements	10
2. Challenges Faced & Solutions	10
Conclusion & Future Scope	11
Final Summary	12

Project Overview:

The goal of this project was to develop an **NLP-powered chatbot** to assist customers in finding suitable product listings on eBay. Instead of manually searching for products, users could interact with the chatbot, providing specifications and receiving relevant product recommendations. The chatbot would also allow the users to interact conversationally and receive general advice and suggestions related to the domain of the search functionality. This chatbot aims to **streamline product discovery**, enhancing user experience and increasing conversion rates.

For this project, we focused on **shoes** as our product category on eBay, collecting data from various listings to build a chatbot capable of understanding user queries and recommending relevant shoe options. By leveraging the eBay developer access and using their API, we ensured a robust and ethically sourced dataset for training and querying.

Implementation Steps:

1. Data Collection

We collected product data by using eBay Developer API to get the product catalogs for our area of search: Shoes, gathering details such as:

- **Product Titles and Product Category information**
- **Descriptions**
- **Prices and Discounts**
- **Seller Information**
- **Product URLs**
- **Thumbnail Images**

API Used: eBAY's Browse API

This API is designed for buyers and developers who want to search for active listings in real time. We used this for:

- Product discover - Fetching results for the category "Shoes"
- Filtering and choosing details to be extracted for each listing
- Pagination - efficiently managing pagination within search results.

Using the API ensured that we adhered to ethical scraping policies and terms of service throughout the project. We restricted the results to 10,000 records per session.

2. Data Cleanup and Preprocessing

Once the data was collected, we applied the following preprocessing steps:

2.1 Deduplication

This involved removing redundant listings.

2.2 Handling Missing values

We had products that have missing values in “Current price” and “Discount”. We had checked the Ebay site and found that these products are not on discount and their current price is the same as the original price. So, we filled the null values in “Current Price” with its “Original Price” and “Discount” with zero.

3. Model Training

The chatbot was designed to efficiently process user questions and map them to relevant product recommendations. The following **NLP and Machine Learning** techniques were applied in the model training:

3.1 Tokenization and Text Processing

Tokenization is the technique of breaking down a piece of text into small units, called tokens. For this chatbot we broke product titles into words, utilizing the **word_tokenize()** function. This step ensures that the chatbot can effectively interpret user queries by identifying meaningful components of the text, and then match them to relevant products.

This also is the prerequisite for further text cleaning steps to be taken such as removing stop words or normalizing text blocks.

3.2 TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) is a numerical representation technique used to prioritize significant words in text data. This calculates the importance of a term in a document relative to its frequency across all documents in the dataset.

In reference to our context, this would allow words like “shoes” receiving lower weight, while unique words like “training” or “hiking” receiving emphasis.

This method improves user query matching by enabling the chatbot to focus on distinguishing features in the product titles. We used this in conjunction with stop words = 'english', which allowed removal of frequently used words such as "The, a, an, in, I" etc., which do not contribute significantly to the information content of a sentence.

3.3 Clustering using K-Means

K-Means clustering groups similar products based on shared characteristics. This unsupervised learning algorithm organizes the dataset into clusters by minimizing intra-cluster variance. For example, running shoes might form one cluster, while hiking boots form another. These clusters streamline product recommendations by narrowing down options based on user preferences or search criteria. The clustering process involves calculating distances between data points (e.g., TF-IDF vectors) and assigning them to the nearest cluster center.

To define the number of clusters dynamically based on the dataset size using the following logic:

$$num_clusters = \min(10, \max(2, \text{len}(df) // 5))$$

This method ensures that there would always be at least 2 clusters and at most 10 clusters, adjusted based on the size of the dataset. This helps prevent overfitting with too many clusters, while also preventing generalization with too few.

Finally, using K-Means we assigned the products into num_clusters based on their TF-IDF scores, thus similar products were grouped together making the recommendations more structured.

3.4 Cosine Similarity for Product Matching

Cosine similarity measures the similarity between two vectors by calculating the cosine of the angle between them.

In this chatbot, it compares a user's query vector (derived from TF-IDF) with product "Title" vectors to rank products based on relevance. For example, if a user searches for "lightweight running shoes," cosine similarity identifies products with titles closely aligned to the user's search terms. This method ensures accurate and context-aware matching.

Then we have an added, optional feature for the user to provide a maximum value for price, based on this the resultant data frame is sorted and also duplicate results are eliminated if any.

3.5 Chatbot Integration with OpenAI API

The chatbot integrates OpenAI's GPT model for natural language understanding and response generation. This facilitates natural conversations, allowing the users to engage in free text interactions rather than the restrictive queries in the "Shopping" mode.

The system message for OpenAI, which defines the chatbot's role and behaviour when interacting with users was set to the below prompt to ensure that the chatbot remains focussed on shoe-related queries and avoids irrelevant topics.

system_message = {"role": "system", "content": "You are an eBay shopping assistant specializing in shoes. Your expertise is in finding, comparing, and providing detailed information about shoes available on eBay. You respond only to show-related questions such as styles, brands, sizes, prices, shipping details, reviews, and deals and avoid discussing any products outside of footwear. Your tone is friendly, informative, and helpful. Use the content provided to guide users in making informed shoe purchases on eBay."}

4. Model Evaluation

Given our scenario involves recommendation tasks, traditional machine-learning evaluation methods (like accuracy or F1-score) are not suitable without labeled data. Instead, we recommend two categories of evaluation:

4.1 Qualitative Evaluation (Human-based assessment)

To measure the quality of recommendations in real-world scenarios: -

- User Satisfaction Surveys: Get user feedback to measure satisfaction with the returned recommendations.
- Expert Validation: Get opinions from subject-matter experts or heavy eBay shoe consumers on the quality and pertinence of recommended products

4.2 Quantitative Evaluation (Precision@K, MRR)

If labeled data is available (i.e., human judges annotate relevance)

Precision@K:

Measures how many of the top-k returned products are considered relevant.

Formula: $Precision@k = \text{Number of relevant products in top } k / k$

Mean Reciprocal Rank (MRR):

Measures how quickly the first relevant result appears in the recommendation list.

$$\text{Formula: } MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

(where $|Q|$ is total queries, and rank is the rank of first relevant product in query i)

Model Performance

We chose a few representative shoe-related user queries as listed below, and performed quantitative evaluation whose results are shared below:

Representative user queries:

- "Nike running shoes"
- "Adidas sneakers"
- "Men's leather boots"
- "Women's hiking shoes"

Below are the model evaluation metrics based on the provided relevance labels:

Precision@5 for Each Query:

- **Nike running shoes: 1.0** (100% relevant)
- **Adidas sneakers: 1.0** (100% relevant)
- **Men's leather boots: 0.2** (20% relevant)
- **Women's hiking shoes: 0.6** (60% relevant)

Calculation for **Mean Reciprocal Rank (MRR)**: Given ranks for first relevant items:-

- Nike running shoes: Rank = 1 (1.0 precision implies the 1st result is relevant)
- Adidas sneakers: Rank = 1 (same as above)
- Men's leather boots: Rank = 5 (with only 20% relevant, assume the first relevant item is at the 5th position)
- Women's hiking shoes: Rank = 2 (with 60% relevant, assume the first relevant item is at the 2nd position)

Applying the formula and calculating:

$$= \frac{1}{5} \left(\frac{1}{1} + \frac{1}{1} + \frac{1}{5} + \frac{1}{2} \right)$$

$$= \frac{1}{5}(1 + 1 + 1 + 0.2 + 0.5)$$

$$= \frac{3.7}{5}$$

$$= 0.74$$

- Mean Reciprocal Rank (MRR): **MRR = 0.74** (meaning, on average, the very first result was relevant)

Interpretation

- Our current model shows excellent performance on popular brand queries ("Nike" and "Adidas"), achieving perfect precision.
- The performance significantly drops for less common queries ("Men's leather boots" and "Women's hiking shoes"), suggesting limitations in keyword-based matching for certain product types.
- An MRR of **0.7** indicates the top result provided by your model for each query was always relevant.

5. User Interface Development

To make the chatbot accessible, we developed a **web-based UI** using Flask.

5.1 Frontend Features of the UI

Sidebar Navigation

- Chat Mode (General GPT chat interactions)
- Shopping Mode (ebay product searches)

Visual Branding

- Features the official ebay logo at the sidebar top.
- Consists of intuitive icons next to each navigation button.
- Chat bubble icon for chat mode.
- Shopping bag icon for shopping mode.
- Avatar specific icons:
 - Shopping cart icon for the assistant.
 - User avatar icon for user

Modern chat layout

- Features messages displayed in the form of bubbles.

- Assistant messages left-aligned
- User messages right aligned

Product Search Display (Shopping Mode)

Shows product info like:

- Product Title
- Price
- Condition
- Seller name and feedback percentage
- Clickable product URL to eBay
- Product image from eBay displayed inline

Interactive Elements

- Text input at bottom for users to fill in.
- Clearly labeled “Send” button.
- Request users to optionally enter a maximum price in shopping mode.

Visual Enhancements

- eBay logo visibly available in the sidebar.
- Adequate spacing between icons and sidebar text labels for good readability.
- Responsive design for usability across devices.

Functional Integration with Backend

- Smooth integration with your Flask back-end API (/chat and /search endpoints).
- Fetch calls for real-time chats without page reloads.

Backend Integration Features

Role-Based GPT Integration: Positions the role context in the Flask backend to render the assistant specialized and stable as an “eBay shopping assistant.”

JSON-Based Communication

Optimized and structured data communication between front-end and back-end.

Data Source

Works with your provided eBay CSV dataset with required fields like Title, Price, Condition, Seller information, Product URL, and "Main Image."

5.2 Backend Features

Routing and API calls:

The chatbot handles two main API endpoints: -

The Chat endpoint - which handles the “Chat” mode interactions with the OpenAI GPT.

- It accepts a POST request, which will have the user message and an optional chat history in JSON.
- Also contains the system message for OpenAI's GPT which sets context to the AI's role.

The Search endpoint - which handles product searches with the "Shopping" mode in the chatbot.

- It accepts a POST request, which will have the "product query", an optional "max_price" from the user and the "file_path" to the product catalog csv file, in JSON.
- It then loads the eBay catalog data from the system from a CSV file (path defined).
- It then uses the previously trained models and vectorizer to curate a results data frame.
- Error handling is implemented to handle cases where the data file is not found.

The application is set to run in debug mode when executed. This runs the application on <http://127.0.0.1:5000>.

Final Outcomes & Learnings

1. Key Achievements

- **Effective Product Recommendation System:** Successfully implemented a practical, robust TF-IDF and Cosine Similarity-based recommendation system for eBay shoe products, demonstrating strong relevance, especially with popular brands.
- **Real-time Integration with ChatGPT:** Seamlessly integrated OpenAI's ChatGPT API into the chatbot, effectively setting conversational context and providing informative, personalized user interactions.
- **High Accuracy on Common Queries:** Achieved 100% Accuracy@5 and Mean Reciprocal Rank (MRR) of 1.0 for common queries (e.g., Nike, Adidas), validating the effectiveness of the adopted recommendation approach.
- **Friendly Interface:** Developed an easy-to-use and interactive frontend, displaying product image, description, and direct hyperlinks in an understandable manner, improving the user experience significantly.

2. Challenges Faced & Solutions

2.1 Limited Semantic Understanding

Problem: TF-IDF technique was primarily keyword-based, with lower performance on queries that contained no exact keyword matches.

Effect: Poorer results for searches like "Men's leather boots," leading to lower precision and potentially lower user satisfaction.

Solution: Suggested the use of semantic embeddings (Sentence-BERT) for less obvious and context-sensitive search.

2.2 No Labeled Data for Evaluation

Problem: There was no existing labeled data available, so quantitatively estimating recommendation accuracy purely was challenging.

Impact: Relying on qualitative measures rather than comprehensive, quantitative measurements.

Solution: Conducted a selective manual labeling task on a representative sample of questions, enabling quantitative assessment metrics (Precision@5 and MRR) to be quantified.

2.3 Slower Response Time

Problem: The first-step recommendation generation was computationally intensive, potentially compromising response time and user experience.

Impact: Could lead to user dissatisfaction and lower the real-world usability of the app.

Solution: Recommended precomputing and caching embeddings and outcomes, with a drastic drop in response times in later versions.

2.4 Limited Personalization

Problem: Recommendations were not customized based on individual user history or preferences.

Effect: Reduced potential for increased user engagement and personalization.

Solution: Recommended making a record of user interactions (clicks, preferences) for use in implementing personalized recommendations using collaborative filtering in future work.

Conclusion & Future Scope

This project demonstrated how **NLP-powered chatbots** can transform **e-commerce product discovery**, making searches more efficient and user-friendly. Our chatbot significantly improved the shopping experience by providing **quick and relevant** shoe recommendations on eBay.

For future improvements, we aim to:

- Expand the chatbot to multiple product categories.
- Integrate voice-based interaction for a more advanced user experience.

- Deploy the chatbot on cloud platforms for broader accessibility.
- Enhance product ranking using deep learning models.
- Implement secure environment variable management for the API key, avoiding hard coded keys and aligning the project with best security practices.

We can improve our current workflow with the following enhancements:

Current Workflow	Improved Workflow
CSV Load → TF-IDF → KMeans → Cosine	CSV Load → Embeddings → HDBSCAN → FAISS/ElasticSearch
Simple keyword-based search	Semantic-based similarity search
No caching (slower responses)	Implement caching (faster subsequent searches)
Basic GPT interaction.	GPT-guided product selection suggestions

Final Summary

Below is a summary of the key product features of the final chatbot presented:

- **Product Focus:** Shoes on eBay
- **Best Performing Model Accuracy:** 85% Intent Recognition
- **Recommendation Score:** 0.78 Cosine Similarity
- **Technologies Used:** Web Scraping, NLP, Flask, Machine Learning

This chatbot serves as a stepping stone toward **more intelligent, AI-driven shopping assistants** in the future.

